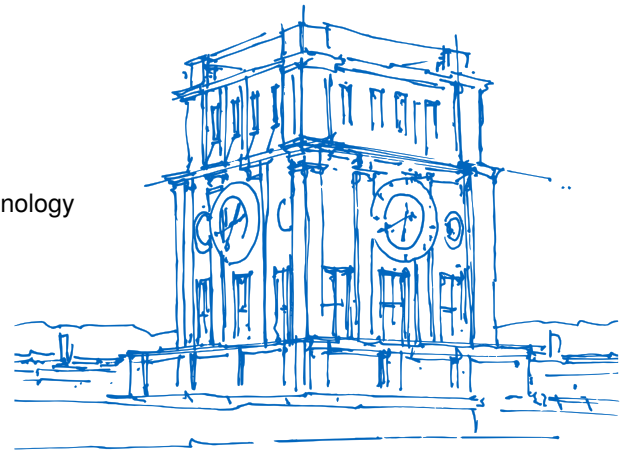
Linux

Überblick und Benutzung

Adrian Stein, Thomas Lindae

School of Computation, Information and Technology
Technische Universität München

29. September 2026



Linux

Von Unix zu Linux

1969 wurde mit dem Unix Betriebssystem bereits wichtige Konzepte umgesetzt:

- Multitasking
- Trennung von Prozessen
- Verwaltung von Zugriffsrechten
- …

Von Unix zu Linux

1969 wurde mit dem Unix Betriebssystem bereits wichtige Konzepte umgesetzt:

- Multitasking
- Trennung von Prozessen
- Verwaltung von Zugriffsrechten
- …

„Unix“ beschreibt heute Systeme die auf Unix aufbauen oder dessen Konzepte nutzen.

Von Unix zu Linux

1969 ·····● *UNIX*—Bell Laboratories

Von Unix zu Linux

1969 · · · · · · ● *UNIX*—Bell Laboratories

1979 · · · · · · ● *UNIX* ist durch (nun) AT&T nicht mehr zu Lehrzwecken zur Verfügung

Von Unix zu Linux

1969 · · · · · · ● *UNIX*—Bell Laboratories

1979 · · · · · · ● *UNIX* ist durch (nun) AT&T nicht mehr zu Lehrzwecken zur Verfügung

1987 · · · · · · ● *MINIX*—Andrew Tanenbaum

Von Unix zu Linux

1969 ····· *UNIX*—Bell Laboratories

1979 ····· *UNIX* ist durch (nun) AT&T nicht mehr zu Lehrzwecken zur Verfügung

1987 ····· *MINIX*—Andrew Tanenbaum

1991 ····· Entwicklung von *Linux* auf Basis von *MINIX*—Linus Torvalds



Linus Torvalds

Von Unix zu Linux

1969 · · · · · · ● *UNIX*—Bell Laboratories

1979 · · · · · · ● *UNIX* ist durch (nun) AT&T nicht mehr zu Lehrzwecken zur Verfügung

1987 · · · · · · ● *MINIX*—Andrew Tanenbaum

1991 · · · · · · ● Entwicklung von *Linux* auf Basis von *MINIX*—Linus Torvalds

1994 · · · · · · ● *Linux* Version 1.0



Linus Torvalds

Verbreitung von Linux

Linux ist das meist verbreitetste Betriebssysteme überhaupt mit vielen Einsatzgebieten:

- Android und ChromeOS
- Personal Computers
- Eingebettete Systeme (Kühlschränke, Router, Medizintechnik, etc.)
- Große Teile der Internet Infrastruktur
- Alle der TOP500 Supercomputer

[1] SuperMUC als Überbegriff für das beste am LRZ gehostete System

Verbreitung von Linux

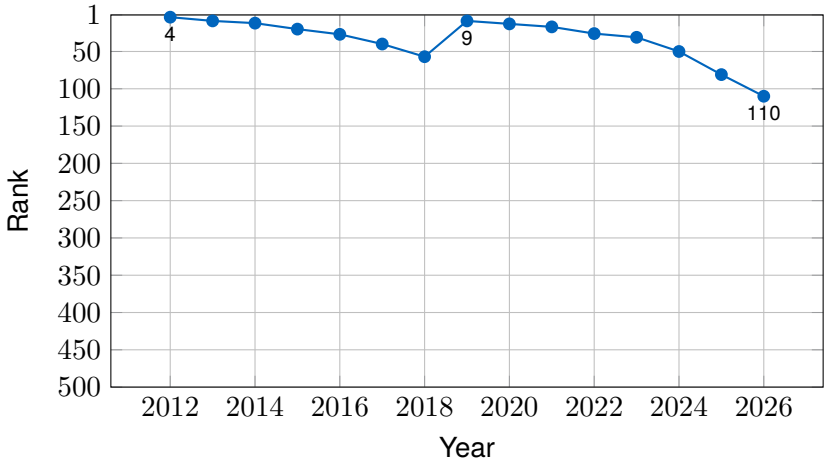
Linux ist das meist verbreitetste Betriebssysteme überhaupt mit vielen Einsatzgebieten:

- Android und ChromeOS
- Personal Computers
- Eingebettete Systeme (Kühlschränke, Router, Medizintechnik, etc.)
- Große Teile der Internet Infrastruktur
- Alle der TOP500 Supercomputer
  - Der SuperMUC[1] (200m vom Hörsaal entfernt) befindet sich auf Platz 110 (Juni 2026)

[1] SuperMUC als Überbegriff für das beste am LRZ gehostete System

Verbreitung von Linux

SuperMUC TOP500



Was ist Linux

Linux ist ein freies („free“) und „Open Source“ Betriebssystem.

Ein Betriebssystem ist die Schnittstelle zwischen Hardware und Software.

Bestandteile eines Betriebssystems (gekürzt):

- **Bootloader**—Verantwortlich für den Systemstart
- **Kernel**—Kern des Betriebssystem („Linux“ beschreibt eigentlich nur den Kernel)
- **Window System**—Ermöglicht graphische Benutzerinterfaces (X, Wayland)
- **Desktop Environment**—Interface für den Nutzer (GNOME, KDE, …)

Free Software vs. Open Source
Free Software

Free Software definiert sich durch 4 Freiheiten:

- Benutzung der Software
- Funktionsweise der Software zu studieren/ändern
- Kopien der Software zu verteilen
- Modifizierte Kopien der Software zu verteilen

➔ „*free*“ wie in *free speech*, nicht wie in *free beer*.



Richard Stallman

Free Software vs. Open Source
Free Software

Free Software definiert sich durch 4 Freiheiten:

- Benutzung der Software
- Funktionsweise der Software zu studieren/ändern
- Kopien der Software zu verteilen
- Modifizierte Kopien der Software zu verteilen

➔ „*free*“ wie in *free speech*, nicht wie in *free beer*.

Die Free Software Foundation (*FSF*), 1983 von Richard Stallman gegründet, vertritt diese 4 Freiheiten (siehe GNU General Public License).



Richard Stallman

Free Software vs. Open Source

Open Source

Open Source wird von der Open Source Initiative (*OSI*) vertreten. Die Werte der *OSI* sind direkt von denen der *FSF* abgeleitet, orientieren sich jedoch ideologisch an den rein praktischen Vorteilen von Quelloffenen Code (Sicherheit, weniger Bugs, etc.).

Free Software vs. Open Source

Open Source

Open Source wird von der Open Source Initiative (*OSI*) vertreten. Die Werte der *OSI* sind direkt von denen der *FSF* abgeleitet, orientieren sich jedoch ideologisch an den rein praktischen Vorteilen von Quelloffenen Code (Sicherheit, weniger Bugs, etc.).

Der Begriff „*Open Source*“ ist oft missverständlich. Die Fähigkeit den Source Code anzusehen bedeutet nicht, dass die Software *Free* oder *Open Source* ist!

Free Software vs. Open Source

Open Source

Open Source wird von der Open Source Initiative (*OSI*) vertreten. Die Werte der *OSI* sind direkt von denen der *FSF* abgeleitet, orientieren sich jedoch ideologisch an den rein praktischen Vorteilen von Quelloffenen Code (Sicherheit, weniger Bugs, etc.).

Der Begriff „*Open Source*“ ist oft missverständlich. Die Fähigkeit den Source Code anzusehen bedeutet nicht, dass die Software *Free* oder *Open Source* ist!

Gängige Lizenzen die unter *Free Software* und *Open Source* fallen:

- GNU General Public License
- Apache License
- FreeBSD License
- Expat License (aka. MIT License)

Linux Distributionen

Ein „Linux“-Betriebssystem ist beschrieben durch eine sog. Distribution („Distro“):

- Ubuntu (Anfängerfreundlich)
  - Pop!_OS
- Fedora (unsere Empfehlung)
- Arch







Linux Distributionen

Im wesentlichen unterscheiden sich Distributionen in den folgenden Aspekten:

- Umfang und Aktualität der verfügbaren Software
- Package Manager
- Installer und (Konfigurations-) Tools
- Wahl des Desktop Environments
- Hardware Unterstützung (z. B. proprietäre Treiber)
- Release Cycle (wie oft gibt es neue Versionen bzw. wie lange werden alte unterstützt)
- Architektur (x86-64 für AMD und Intel, Apple Silicon, RISC-V, …)
- Dokumentation
- Support
- Lizenzierung

Linux über Virtual Machines

Virtual Machines können ein weiteres Betriebssystem auf dem „Host“-System ausführen.

Dieses weitere Betriebssystem ist:

- in der Bedienung ähnlich zu „normalen“ Programmen
- vollkommen abgetrennt vom „Host“-System
- oft langsamer als wenn es direkt installiert ist

Linux über Virtual Machines

Virtual Machines können ein weiteres Betriebssystem auf dem „Host“-System ausführen.

Dieses weitere Betriebssystem ist:

- in der Bedienung ähnlich zu „normalen“ Programmen
- vollkommen abgetrennt vom „Host“-System
- oft langsamer als wenn es direkt installiert ist

Virtual Machines:

- Virtual Box
- QEMU (ermöglicht auch Emulation anderer Architekturen, z. B. RISC-V, ARM, …)

➔ Virtual Machines eignen sich um verschiedene Linux Distributionen mal auszuprobieren!

Windows Subsystem for Linux (WSL)

Das WSL ermöglicht die Nutzung eines Linux Systems direkt auf einem Windows Computer.

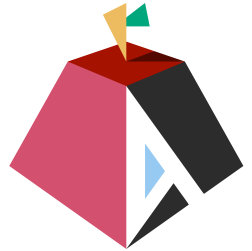
Die Windows Nutzer unter Ihnen sollten dieses bereits installiert haben (siehe *Übung 0*)!

Installation mithilfe dem Guide von Microsoft.

Linux auf Apple Silicon

Asahi Linux macht Linux auf Apple Silicon möglich!

- Fedora Asahi Remix für die beste Erfahrung
- Manche Features sind teilweise eingeschränkt
- Kann neben MacOS im Dual-Boot benutzt werden



Das Filesystem

Filesysteme

Ein Filesystem dient der organisation von Dateien auf Speicher-Medien (HDD, SSD, …).

Es gibt eine vielzahl an Filesystemen mit unterschiedlichen Features/Limitationen:

Filesysteme

Ein Filesystem dient der organisation von Dateien auf Speicher-Medien (HDD, SSD, …).

Es gibt eine vielzahl an Filesystemen mit unterschiedlichen Features/Limitationen:

- Linux typisch:
  - ext4
  - Btrfs

Filesysteme

Ein Filesystem dient der organisation von Dateien auf Speicher-Medien (HDD, SSD, …).

Es gibt eine vielzahl an Filesystemen mit unterschiedlichen Features/Limitationen:

- Linux typisch:
  - ext4
  - Btrfs
- Windows typisch:
  - FAT32
  - exFAT
  - NTFS

Filesysteme

Ein Filesystem dient der organisation von Dateien auf Speicher-Medien (HDD, SSD, …).

Es gibt eine vielzahl an Filesystemen mit unterschiedlichen Features/Limitationen:

- Linux typisch:
  - ext4
  - Btrfs
- Windows typisch:
  - FAT32
  - exFAT
  - NTFS
- MacOS typisch:
  - APFS
  - HFS+

Filesysteme

Nicht jedes Filesystem wird automatisch von jedem Betriebssystem erkannt.

Übersicht:

- NTFS—Fedora und Ubuntu kommen mit Treibern; MacOS kann Dateien nur lesen
- exFAT—Wird vom Linux Kernel unterstützt; MacOS volle unterstützung
- ext4—Auf Windows und MacOS nur mit zusätzlichen Treibern möglich

Filesystem Pfade

Über Dateipfade kann man u. A.:

- leichter durch das Filesystem navigieren
- die Ein und Ausgaben von Programmen spezifizieren

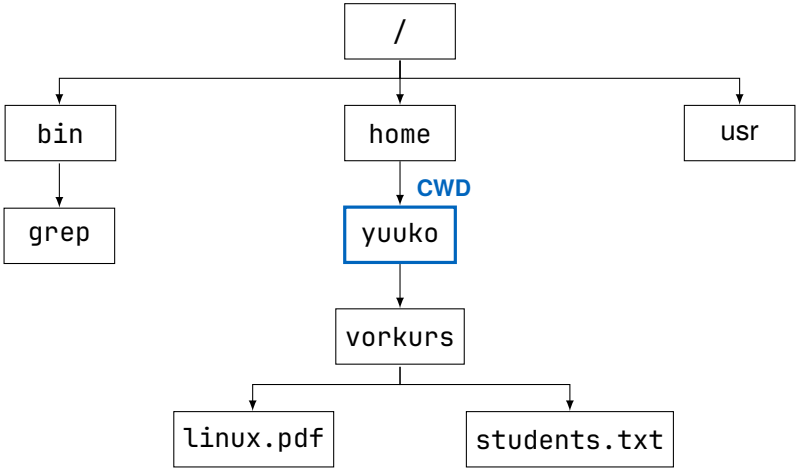
Pfade sind:

- „case-sensitive“ (`file` $\neq$ `File`)
- Abschnitte (Ordner) eines Pfades werden durch `/` getrennt

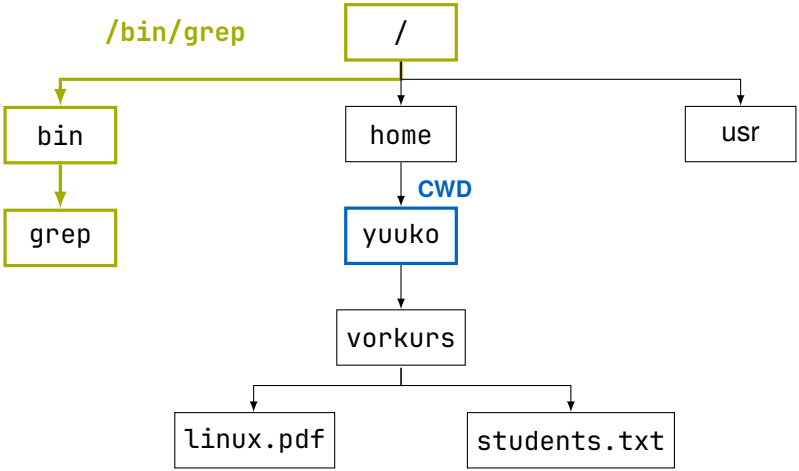
Bei Dateipfaden wird zwischen *Absoluten* und *Relativen* Pfaden unterschieden:

- Absolute Pfade beginnen bei der Dateisystem Root („`/`“):
  `/bin/grep`
- Relative Pfade sind relativ zum *CWD* („current working directory“):
  `vorkurs/linux.pdf`

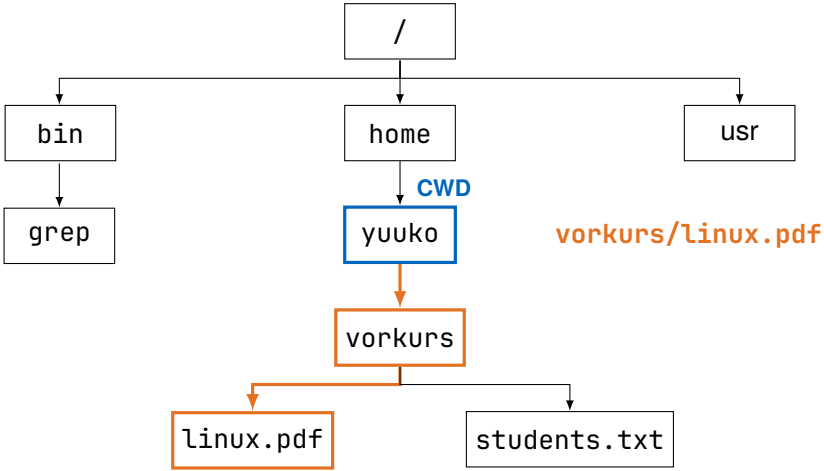
Filesystem Pfade



Filesystem Pfade



Filesystem Pfade



Filesystem Pfade

Bedienhilfen (Aktuelles *CWD* ist `/home/yuuko` für die Beispiele):

- Mit `.` wird das aktuelle Verzeichnis referenziert
  `./vorkurs/linux.pdf` ist das gleiche wie `vorkurs/linux.pdf`
- Mit `..` wird das Oberverzeichnis referenziert
  `../yuuko/vorkurs/linux.pdf` ist das gleiche wie `vorkurs/linux.pdf`
- Mit ~ wird das Home-Verzeichnis referenziert (z. B. `/home/yuuko`)

Jeder Nutzer bekommt ein eigenes Home-Verzeichnis `/home/<username>` mit:

- default Verzeichnissen wie `Documents/`, `Downloads/`, …
- nutzerspezifischen Konfigurationsdateien (oft gesammelt in `.config/`)

Filesystem Pfade

Datei- und Ordnernamen mit Leerzeichen müssen mit `"` oder `'` geklammert werden:

- `vorkurs/"ordner mit spaces"/datei.txt`
- `vorkurs/'ordner mit spaces'/datei.txt`

Unterschiede zu anderen Betriebssystemen:

- MacOS basiert wie Linux auf Unix. Dadurch sind die Unterschiede recht gering.
- Windows nutzt `\` anstelle von `/`
- Windows benötigt bei absoluten Pfaden sog. Drive-Letters[2]: `C:\Users\<username>`

[2] Drive-Letters sind mit einem physischen Datenträger assoziiert

Das Terminal

Was ist das Terminal

Das Terminal als rudimentärste Interaktionsmöglichkeit (historisch ein eigenes Gerät).



Sperry Rand UNIVAC Uniscope 100

Was ist das Terminal

Das Terminal als rudimentärste Interaktionsmöglichkeit (historisch ein eigenes Gerät).



Sperry Rand UNIVAC Uniscope 100

Im Terminal werden:

- Commands zur Ausführung eingegeben
- Die Ergebnisse von Commands angezeigt
- Im Terminal kann fast alles gemacht werden:
  - Textverarbeitung
  - Programmieren
  - E-Mail
  - Star Wars anschauen
  - …

Was ist das Terminal

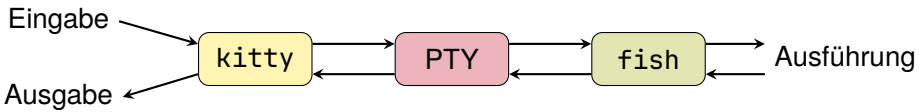
Wenn man *heute* von „Terminal“ redet, meint man eine Kombination von drei Dingen:

1. Terminal Emulator
   - Liest Eingabe (Tastatur) und Zeigt Ergebnisse an (Bildschirm)
   - z.B. kitty, Alacritty, Ghostty, …
2. PTY/TTY ((Pseudo-) Teletypewriter)
   - Kommunikation zwischen Emulator und Shell
3. Shell
   - Interpretiert die eingegebenen Commands
   - z. B. sh, bash, zsh, fish, …
     PowerShell, …

Was ist das Terminal

Wenn man *heute* von „Terminal“ redet, meint man eine Kombination von drei Dingen:

1. Terminal Emulator
   - Liest Eingabe (Tastatur) und Zeigt Ergebnisse an (Bildschirm)
   - z.B. kitty, Alacritty, Ghostty, …
2. PTY/TTY ((Pseudo-) Teletypewriter)
   - Kommunikation zwischen Emulator und Shell
3. Shell
   - Interpretiert die eingegebenen Commands
   - z. B. sh, bash, zsh, fish, …
     PowerShell, …



Programme und Ein/Ausgaben

Programme und Ein/Ausgaben

- **Environment Variablen**:
  Jede Shell hat anpassbare Variablen die von Programmen gelesen werden können

Programme und Ein/Ausgaben

- **Environment Variablen**:
  Jede Shell hat anpassbare Variablen die von Programmen gelesen werden können
- **CLI Arguments**:
  Einmalige übergabe von Parametern & Eingaben bei Programmstart

Programme und Ein/Ausgaben

- **Environment Variablen**:
  Jede Shell hat anpassbare Variablen die von Programmen gelesen werden können
- **CLI Arguments**:
  Einmalige übergabe von Parametern & Eingaben bei Programmstart
- **stdin**:
  Input-Stream an das Programm (z. B. Tastatur Eingaben)

Programme und Ein/Ausgaben

- **Environment Variablen**:
  Jede Shell hat anpassbare Variablen die von Programmen gelesen werden können
- **CLI Arguments**:
  Einmalige übergabe von Parametern & Eingaben bei Programmstart
- **stdin**:
  Input-Stream an das Programm (z. B. Tastatur Eingaben)
- **stdout**:
  Output-Stream des Programms (wird i. d. R. vom Terminal-Emulator angezeigt)

Programme und Ein/Ausgaben

- **Environment Variablen**:
  Jede Shell hat anpassbare Variablen die von Programmen gelesen werden können
- **CLI Arguments**:
  Einmalige übergabe von Parametern & Eingaben bei Programmstart
- **stdin**:
  Input-Stream an das Programm (z. B. Tastatur Eingaben)
- **stdout**:
  Output-Stream des Programms (wird i. d. R. vom Terminal-Emulator angezeigt)
- **stderr**:
  Wie **stdout** aber speziell für Fehler-Nachrichten vom Programm

Programme und Ein/Ausgaben

- **Environment Variablen**:
  Jede Shell hat anpassbare Variablen die von Programmen gelesen werden können
- **CLI Arguments**:
  Einmalige übergabe von Parametern & Eingaben bei Programmstart
- **stdin**:
  Input-Stream an das Programm (z. B. Tastatur Eingaben)
- **stdout**:
  Output-Stream des Programms (wird i. d. R. vom Terminal-Emulator angezeigt)
- **stderr**:
  Wie **stdout** aber speziell für Fehler-Nachrichten vom Programm
- **…**

Programme und Ein/Ausgaben

Environment Variables

Environment Variables können von Prozessen über deren „Umgebung“ gelesen werden.

Env. Variable	Beschreibung
`HOSTTYPE`	Architektur des Computers (z. B. *x86-64*)
`HOME`	Pfad zu `/home/<username>`
`PWD`	Pfad zum aktuellen *working directory*
`PATH`	Pfade zu Ordnern mit ausführbaren Programmen
`EDITOR`	Default Terminal Text Editor

Programme und Ein/Ausgaben

Environment Variables

Environment Variables können von Prozessen über deren „Umgebung“ gelesen werden.

Env. Variable	Beschreibung
`HOSTTYPE`	Architektur des Computers (z. B. *x86-64*)
`HOME`	Pfad zu `/home/<username>`
`PWD`	Pfad zum aktuellen *working directory*
`PATH`	Pfade zu Ordnern mit ausführbaren Programmen
`EDITOR`	Default Terminal Text Editor

Um ein Programm zu starten muss dem Kernel der vollständige Pfad übergeben werden. Wird nur der Name des Programms benutzt, werden die Ordner in `PATH` durchsucht. Ein Programm im aktuellen Ordner kann einfach mit `./program` ausgeführt werden!

Programme und Ein/Ausgaben
CLI Arguments—Typische Konventionen

```
command [OPTIONS...] arguments
```

- `command`—Das auszuführende Command
- `[OPTIONS...]`—Optionen für das Command-Verhalten
  - Unterscheidung in *Long-Option* (z. B. `--version`) und *Short-Option* (z. B. `-v`)
  - Manche *Options* nehmen auch selbst weitere Argumente
  - *Options* ohne *arguments* werden auch *Flags* genannt
  - Nützliche *Flags*: `--help` (bzw. `-h`) und `--version` (bzw. `-v`)
- `arguments`—Weitere Argumente (z. B. Dateien, Pfade, konkrete Eingaben, …)

Programme und Ein/Ausgaben
CLI Arguments—Typische Konventionen

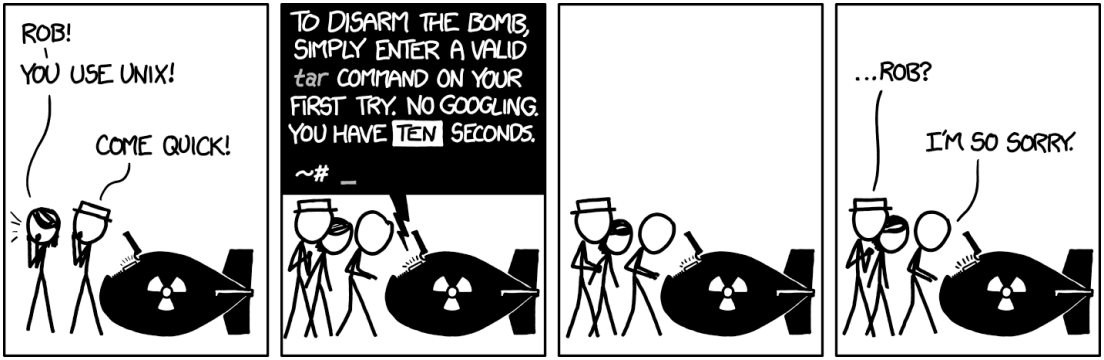
```
command [OPTIONS...] arguments
```

- `command`—Das auszuführende Command
- `[OPTIONS...]`—Optionen für das Command-Verhalten
  - Unterscheidung in *Long-Option* (z. B. `--version`) und *Short-Option* (z. B. `-v`)
  - Manche *Options* nehmen auch selbst weitere Argumente
  - *Options* ohne *arguments* werden auch *Flags* genannt
  - Nützliche *Flags*: `--help` (bzw. `-h`) und `--version` (bzw. `-v`)
- `arguments`—Weitere Argumente (z. B. Dateien, Pfade, konkrete Eingaben, …)

Leider ist nicht jedes Command konform mit dieser Darstellung!

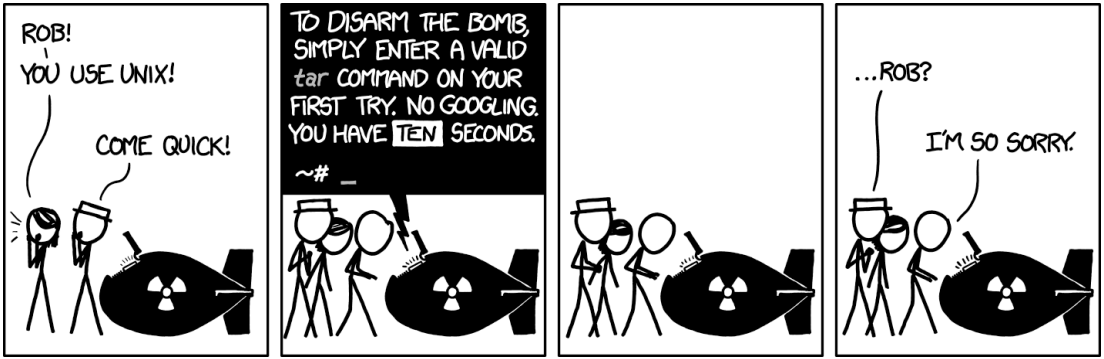
Essentielle Commands

Essentielle Commands



xkcd 1168

Essentielle Commands



xkcd 1168

Man muss nicht alle Commands auswendig kennen (kann aber praktisch sein)!

Essentielle Commands | `man`

Steht für „manual“.

Ruft die Bedienungsanleitung des angegebenen Objekts auf (z. B. ein Programm).

Typische Nutzung:

- `man <command>` (z. B. `man man`)

Essentielle Commands | `ls`

Steht für „list“.

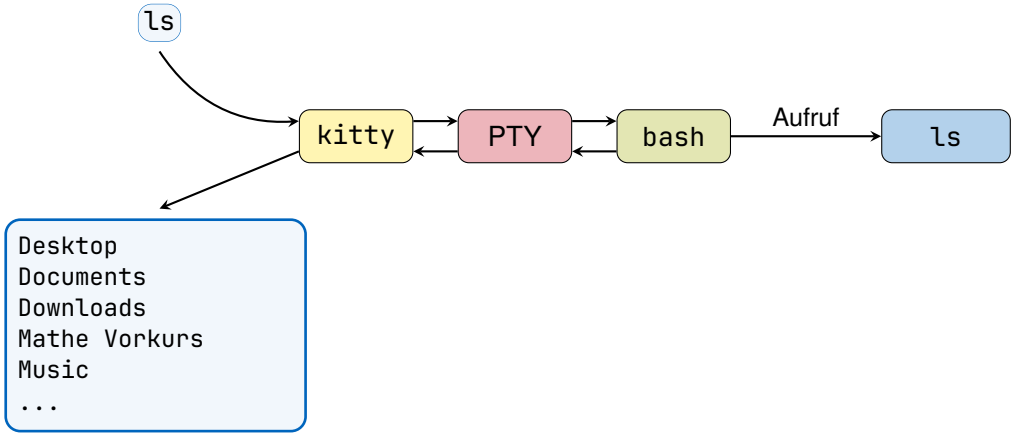
Auflistung des Ordner Inhalts.

Typische Nutzung:

- `ls`
- `ls path/to/folder`
- `ls -a`—Zeigt auch hidden files (Dateien/Ordner die mit `.` anfangen) an
- `ls -l`—Zeigt Zusatzinformationen an (Typ, Nutzungsrechte, Größe, Datum)

Essentielle Commands | `ls`

Beispiel: Von Ein- zu Ausgabe



Essentielle Commands | `cd`

Steht für „change directory“.

Wechsel des *current working directory*.

Typische Nutzung:

- `cd path/to/folder`
- `cd ..`—Wechsel in den Parent-Ordner
- `cd ~`—Wechsel zum *Home* Verzeichnis

Essentielle Commands | `cd`

Steht für „change directory“.

Wechsel des *current working directory*.

Typische Nutzung:

- `cd path/to/folder`
- `cd ..` —Wechsel in den Parent-Ordner
- `cd ~` —Wechsel zum *Home* Verzeichnis

Bei `cd` handelt es sich nicht um ein Programm, sondern um ein sog. „builtin“ der Shell.

Essentielle Commands | `pwd`

Steht für „print working directory“.

Zeigt das *current working directory* an.

Typische Nutzung:

- `pwd`

Essentielle Commands | `cat`

Steht für „concatenate files and print“.

Gibt Dateien auf der Ausgabe aus.

Typische Nutzung:

- `cat test.txt`—Ausgabe der Datei `test.txt`
- `cat one.txt two.txt`—Gibt die Dateien `one.txt` und `two.txt` aus

Essentielle Commands | `echo`

Gibt den angegebenen Text wieder aus.

Typische Nutzung:

- `echo "beliebiger Text"`

Essentielle Commands | `mkdir`

Steht für „make directory“.

Erstellt Ordner.

Typische Nutzung:

- `mkdir folder`—Erstellt den Ordner `folder` im aktuellen *CWD*
- `mkdir f1 f2 f3`—Erstellt die Ordner `f1`, `f2` und `f3` im aktuellen *CWD*

Essentielle Commands | `rm`

Steht für „remove“.

Löscht Ordner und Dateien.

Typische Nutzung:

- `rm test.txt`
- `rm -d empty-folder`
- `rm -r nonempty-folder`—Löschen eines nicht-leeren Ordners und dessen Inhalt
- `rm -f test.txt`—Erzwingt Löschung und ignoriert nicht existente Dateien

Essentielle Commands | `touch`

Änderung der Zeitstempel einer Datei oder eines Ordners.

Typische Nutzung:

- `touch test.txt`—Aktualisiert den Zeitstempel bzw. erstellt die Datei `test.txt`

Essentielle Commands | `mv`

Steht für „move“.

Verschiebung oder Umbenennung von Dateien.

Typische Nutzung:

- `mv path/to/file new/path/` —Verschiebt `file` von `path/to/` nach `new/path/`
- `mv path/to/file new/path/elif` —Wie oben aber mit Umbenennung der Datei zu `elif`

Essentielle Commands | `cp`

Steht für „copy“.

Kopieren von Dateien und Ordnern.

Typische Nutzung:

- `cp path/to/file other-path/to/copy` —Kopiert `file` zu `other-path/to/copy`

Essentielle Commands | `sudo`

Steht für „su-do“ („substitute user do“, veraltet auch „superuser do“).

Ausführen eines Commands mit den Rechten eines anderen Nutzers (z. B. *Root*[3]).

`sudo` ist für jedes Command welches das System ändert nötig:

- `sudo pacman -S terminator`[4] (Installation von Software „ändert“ das System)

[3] *Root* beschreibt die höchstmöglichen Rechte des Systems

[4] *pacman* ist der Package Manager von Arch Linux (mehr dazu später)

POSIX

POSIX

POSIX (**Portable Operating System Interface**)[5] definiert Unix „artige“ Systeme:

- Programmierschnittstellen (z. B. *syscalls*)
- Existenz bestimmter System-Programme
- Shell „Sprache“ (Syntax, Features, Reserved Words, Variablen, „Built-In“ Utilities, …)
- …

[5] POSIX Standard

[6] Die meisten „simplen“ POSIX Shell-Operatoren werden oft auch von nicht POSIX Shells unterstützt!

POSIX

POSIX (**Portable Operating System Interface**)[5] definiert Unix „artige“ Systeme:

- Programmierschnittstellen (z. B. *syscalls*)
- Existenz bestimmter System-Programme
- Shell „Sprache“ (Syntax, Features, Reserved Words, Variablen, „Built-In“ Utilities, …)
- …

➔ POSIX compliant Programme/Scripte können in jeder POSIX *Umgebung* benutzt werden!

[5] POSIX Standard

[6] Die meisten „simplen“ POSIX Shell-Operatoren werden oft auch von nicht POSIX Shells unterstützt!

POSIX

POSIX (**Portable Operating System Interface**)[5] definiert Unix „artige“ Systeme:

- Programmierschnittstellen (z. B. *syscalls*)
- Existenz bestimmter System-Programme
- Shell „Sprache“ (Syntax, Features, Reserved Words, Variablen, „Built-In“ Utilities, …)
- …

➔ POSIX compliant Programme/Scripte können in jeder POSIX *Umgebung* benutzt werden!

Beispiel Shells:

- POSIX: sh, bash
- Nicht POSIX[6]: zsh, fish, powershell

[5] POSIX Standard

[6] Die meisten „simplen“ POSIX Shell-Operatoren werden oft auch von nicht POSIX Shells unterstützt!

POSIX
Control Operators

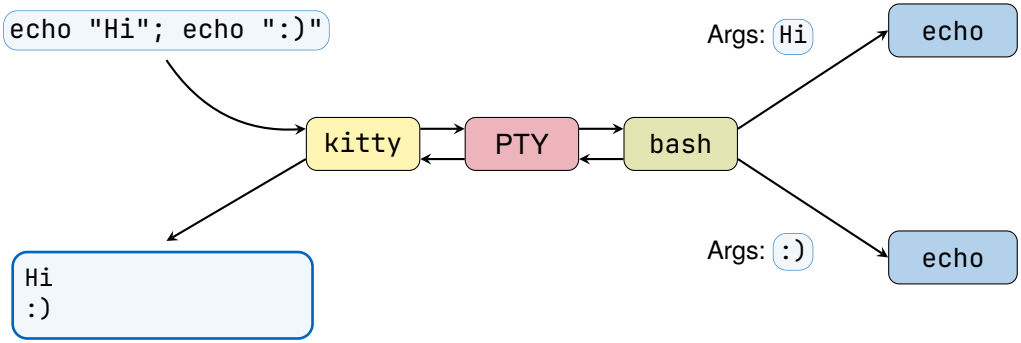
Control Operators (`;` `&&`, `||`, `&`, `(` und `)`, `|`) modifizieren die Ausführung von Commands.

`;`

- Sequentielle Ausführung mehrerer Commands
- Bsp.: `echo "Hi"; echo ":)"; echo "!"`

Die Shell interpretiert die Eingabe und setzt die spezifizierten Programmaufraufe um.

POSIX
Control Operators (Beispiel 1)



POSIX
Control Operators

`&&`

- „AND“-Verknüpfung: Das zweite wird ausgeführt wenn das erste **erfolgreich** war
- Bsp.: `cd MaybeExists && ls`

POSIX
Control Operators

`&&`

- „AND“-Verknüpfung: Das zweite wird ausgeführt wenn das erste **erfolgreich** war
- Bsp.: `cd MaybeExists && ls`

`||`

- „OR“-Verknüpfung: Letzteres wird nur ausgeführt wenn ersteres **fehlgeschlagen** ist
- Bsp.: `cd MaybeExists || echo "Folder does not exist!"`

POSIX
Control Operators

`&`

- Das vorangestellte Command wird im Hintergrund ausgeführt
- Bsp.: `sleep 10 &`

[7] weitere sind z. B. `{` und `}`, mit denen Commands innerhalb der „Haupt“-Shell gruppiert werden

POSIX
Control Operators

`&`

- Das vorangestellte Command wird im Hintergrund ausgeführt
- Bsp.: `sleep 10 &`

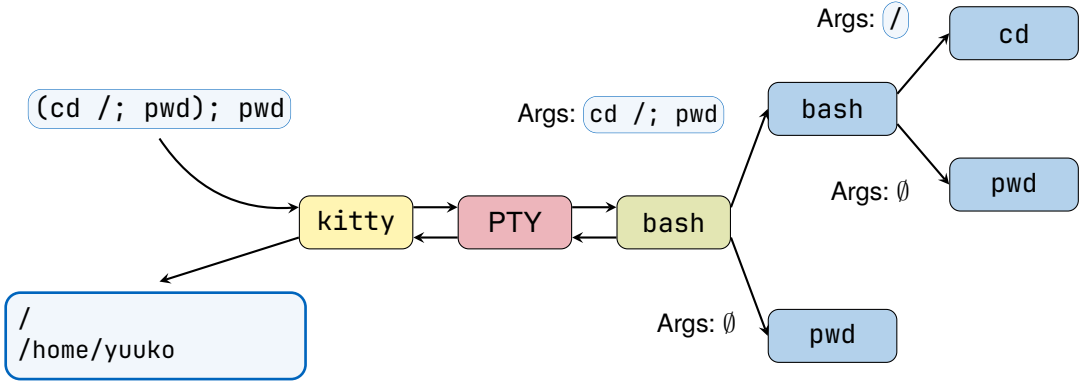
`(` und `)`

- Geklammertes wird in einer „Sub“-Shell unabhängig von der „Haupt“-Shell ausgeführt
- Bsp.: `(cd /; pwd); pwd`
- Gehört auch zu den *Grouping Commands*[7] (Untergruppe der *Compound Commands*)

[7] weitere sind z. B. `{` und `}`, mit denen Commands innerhalb der „Haupt“-Shell gruppiert werden

POSIX
Control Operators (Beispiel 2)

Sei der *agierende* Nutzer (*yuuko*) gerade in `~`:



POSIX
Control Operators

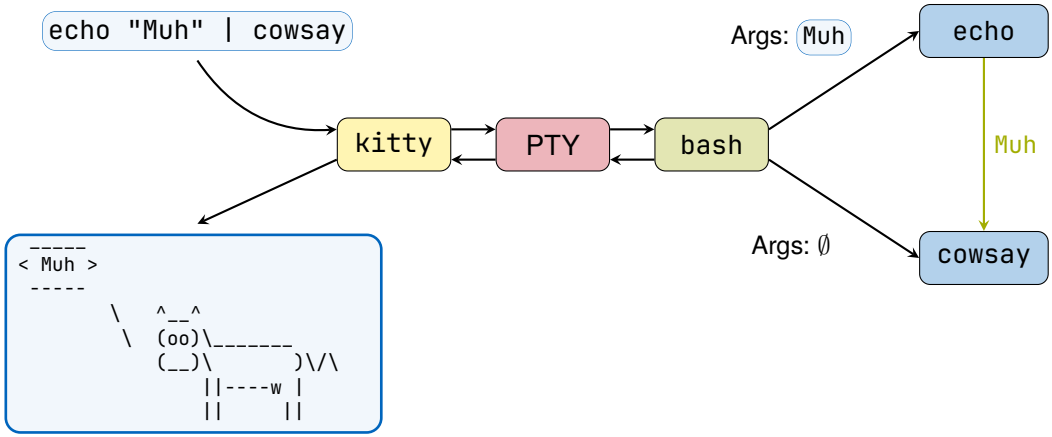
`|` (genannt „Pipe“)

- Verbindet **stdout** (Ausgabe) des ersten Programms mit **stdin** (Eingabe) des zweiten
- Bsp.: `echo "Muh" | cowsay`[8]

[8] Probieren Sie gerne auch mal: `fortune | cowsay`

POSIX

Control Operators (Beispiel 3)



POSIX
Redirection Operators

Redirection Operators (z. B. `>`, `>>`, `<`, `<<`) ermöglichen redirektionen von Daten.

`>` und `>>`

- Umleitung von *stdout* in eine Datei. `>` löscht den Dateiinhalt vor dem schreiben
- Bsp.: `echo "HelloWorld!" > greeting.txt` bzw.
  `echo "HelloWorld!" >> greeting.txt`

[9] Manche Programme unterstützen sowohl CLI Arguments als auch *stdin* für ihre Eingaben!

POSIX
Redirection Operators

Redirection Operators (z. B. `>`, `>>`, `<`, `<<`) ermöglichen redirektionen von Daten.

`>` und `>>`

- Umleitung von *stdout* in eine Datei. `>` löscht den Dateiinhalt vor dem schreiben
- Bsp.: `echo "HelloWorld!" > greeting.txt` bzw.
  `echo "HelloWorld!" >> greeting.txt`

`<`

- Schreiben des gesamten Dateiinhalts an *stdin*
- Bsp.: `cowsay < meaning-of-life.txt`[9]

[9] Manche Programme unterstützen sowohl CLI Arguments als auch *stdin* für ihre Eingaben!

POSIX
Redirection Operators

`<<`

- Wie `<`, bloß wird der Inhalt von einem *Here-Document* genommen
- *Here-Document* als Ad-Hoc Datei (beliebige Delimiter, z. B. `EOF`, „klammern“ den Inhalt)

POSIX
Redirection Operators

`<<`

- Wie `<`, bloß wird der Inhalt von einem *Here-Document* genommen
- *Here-Document* als Ad-Hoc Datei (beliebige Delimiter, z. B. `EOF`, „klammern“ den Inhalt)

```
cowsay <<EOF
Such yummy grass
NomNomNom
EOF
```

```
cowsay <<EOF1; cowsay <<EOF2
Muh
EOF1
You said it!
EOF2
```

Terminal—Tipps und Tricks

Terminal—Tipps und Tricks

Bedienhilfen[10]:

- Aufruf vorheriger Befehle durch „Pfeiltaste nach oben“
- Angefangene Pfade können oft mit **Tab** vervollständigt werden
- Löschen der aktuellen Eingabe mit **Strg+U**
- Abbrechen eines laufenden Befehls mit **Strg+C**
- Kopieren aus dem Terminal mit **Strg+Shift+C**
- Kopieren in das Terminal mit **Strg+Shift+V**
- „Clean“ Terminal mit **Strg+L** (Terminal History bleibt bestehen)
- „Clean“ Terminal mit dem Command `clear` (Terminal History wird gelöscht)

[10] sollten i. d. R. überall funktionieren

Package Management

Package Management

Package Manager verwalten installierte Software. Dazu gehört u. A.:

- Installation
- Aktualisierung
- Deinstallation
- Überprüfung von Software Dependencies

Package Management

Package Manager verwalten installierte Software. Dazu gehört u. A.:

- Installation
- Aktualisierung
- Deinstallation
- Überprüfung von Software Dependencies

Durch Package Manager wird die Installation von Software erleichtert:

- Jegliche Software wird über die gleiche Methode installiert
- Konflikte zwischen Software Paketen werden erkannt und verhindert
- Keine seperaten Installer die:
  - zusätzliche Software installieren und Werbung zeigen
  - über nicht vertrauenswürdige Webseiten herunterzuladen sind

Package Management

Package Manager installieren Software Pakete aus sog. *Repositories*.

Jede Linux Distribution hat einen eigenen Package Manager und zugehörige Repositories.

- Ubuntu: apt
- Fedora: dnf
- Arch: pacman
- OpenSUSE: Zypper
- Und viele mehr! (xbps, eopkg, yum, …)

Package Management

Distributionsübergreifende Package Manager die zusätzlich installiert werden können:

- Snap
- Flatpak
- Homebrew
- Nix

Vorteile von zusätzlichen Package Managern:

- Installation von Software die nicht beim „Host“ Package Managers erhältlich sind
- Andere Software Versionen
- Package Manager spezifische vorteile (z. B. Cross-Platform, Sandboxing, …)

Package Management

Package Manager können auch unter Windows und MacOS genutzt werden.

Windows:

- winget (Offiziell von Microsoft)
- scoop
- chocolatey

MacOS:

- Homebrew
- MacPorts
- Nix

SSH

SSH

SSH steht für *Secure Shell*:

- Ermöglicht sichere Verbindungen in ungesicherten Netzwerken (z. B. Internet)
- Dient der (remote) Systemverwaltung und Nutzung

[11] Dient der Sicherstellung dass man mit dem Service verbunden ist, mit dem man tatsächlich kommunizieren will!

SSH

SSH steht für *Secure Shell*:

- Ermöglicht sichere Verbindungen in ungesicherten Netzwerken (z. B. Internet)
- Dient der (remote) Systemverwaltung und Nutzung

SSH Verbindung zur Rechnerhalle

```
ssh <ito-username>@lxhalle.in.tum.de
```

➔ Authentifizierung (per default) über das `<ito-password>`
➔ Verbindung kann mit `exit` beendet werden!

Bei der ersten Verbindung sollte man die Fingerprints der Rechnerhalle verifizieren.[11]

[11] Dient der Sicherstellung dass man mit dem Service verbunden ist, mit dem man tatsächlich kommunizieren will!

SSH Keys

SSH-Keys ermöglichen ein hohes Sicherheitsniveau für recht geringen Aufwand und sind daher der Authentifizierung mit Passwort vorzuziehen![12]

Generierung:

- `ssh-keygen -t rsa -b 4096` für ein *RSA* (Rivest-Shamir-Adleman) Schlüsselpaar
- `ssh-keygen -t ed25519` für ein *ECC* (Elliptic Curve Cryptography) Schlüsselpaar

Ein Schlüsselpaar besteht aus einem:

- öffentlichen Schlüssel (z. B. `~/.shh/id_ed25519.pub`)
- privaten Schlüssel (z. B. `~/.ssh/id_ed25519`)

[12] Manche Server verbieten Remote-Login per Passwort gänzlich

SSH Keys

Umgang mit einem SSH Schlüsselpaar:

- Der *öffentliche Schlüssel* darf weitergegeben werden[13]
- Der *private Schlüssel* sollte *niemals* ihren Rechner verlassen!
- Bei der Erstellung immer eine gute Passphrase wählen
  ➔ **Achtung**: Von einer leeren Passphrase ist abzuraten!

Installation eines SSH Keys auf der Rechnerhalle

```
ssh-copy-id -i ~/.ssh/<key>.pub <ito-username>@lxhalle.in.tum.de
```

Bei umbenannten Schlüsseln ist oft eine *SSH-Config* nötig (`~/.ssh/config`)!

[13] z. B. an die Rechnerhalle, LRZ, GitHub, …

Access Control

Access Control
Generelle Konzepte

Verwaltung von Nutzungsrechten ist wichtig aber auch schwierig:

- Dateien unterliegen oft verschiedenen „Geheimhaltungsgraden“
  ➔ Privates, Programmieraufgaben, Klausuren, …
- Oft interagieren viele Leute mit dem gleichen System
  ➔ Schulen, Universitäten, Unternehmen, …

Access Control
Generelle Konzepte

Verwaltung von Nutzungsrechten ist wichtig aber auch schwierig:

- Dateien unterliegen oft verschiedenen „Geheimhaltungsgraden“
  ➔ Privates, Programmieraufgaben, Klausuren, …
- Oft interagieren viele Leute mit dem gleichen System
  ➔ Schulen, Universitäten, Unternehmen, …

In Linux werden Zugriffsrechte üblichwerweise über *Access-Control-Lists* (*ACL*) gemanaged.

Access Control

- *ACL*: Jedes Objekt (z. B. Dateien, Ordner, …) speichert eigene *Permissions*
- *Permissions* werden für **Owner**, **Group** und **Other** spezifiziert[14]
- Folgende *Permissions* existieren:
  - **R**ead (Bei Ordnern kann damit der Inhalt gelistet werden)
  - **W**rite (Bei Ordnern dürfen Dateien angelegt/gelöscht werden)
  - e**X**ecute (Bei Ordnern wird Traversierung und Zugriff auf „Metadaten“ erlaubt[15])

[14] Rechte Zuweisung nach „first fit“. Sind Sie der *Owner* ist es egal ob *Other* mehr kann!

[15] Fehlende **x** Rechte auf einem Ordner können teilweise die anderen Rechte Einschränken!

Access Control

- *ACL*: Jedes Objekt (z. B. Dateien, Ordner, …) speichert eigene *Permissions*
- *Permissions* werden für **Owner**, **Group** und **Other** spezifiziert[14]
- Folgende *Permissions* existieren:
  - **R**ead (Bei Ordnern kann damit der Inhalt gelistet werden)
  - **W**rite (Bei Ordnern dürfen Dateien angelegt/gelöscht werden)
  - e**X**ecute (Bei Ordnern wird Traversierung und Zugriff auf „Metadaten“ erlaubt[15])

```
ls -l
```

```
-rw-r--r-- 1 mlutt students  2048 Sep 13 14:09 Exercise-1.pdf
drwxr-xr-x 2 mlutt tutors   60160 Sep 17 14:09 Mathe-Vorlesung
-rw-r----- 1 mlutt tutors    4096 Sep 14 13:37 Solution-1.pdf
```

[14] Rechte Zuweisung nach „first fit“. Sind Sie der *Owner* ist es egal ob *Other* mehr kann!

[15] Fehlende **x** Rechte auf einem Ordner können teilweise die anderen Rechte Einschränken!

Access Control
Anpassungen mit `chmod`

Der Owner eines Datum kann mit `chmod` (change File-Mode bits) die Zugriffsrechte ändern.

Rechte	Binär	Oktal
Read (**r**)	`100`	`4`
Write (**w**)	`010`	`2`
Execute (**x**)	`001`	`1`
r & **x**[16]	`101`	`5`

[16] Beliebige Kombinationen der Permission-Bits (**rwx**) sind natürlich Möglich

Access Control
Anpassungen mit `chmod`

Der Owner eines Datum kann mit `chmod` (change File-Mode bits) die Zugriffsrechte ändern.

Rechte	Binär	Oktal
Read (**r**)	`100`	`4`
Write (**w**)	`010`	`2`
Execute (**x**)	`001`	`1`
r & **x**[16]	`101`	`5`

Rechte für *Owner*, *Group* und *Other* wird explizit durch ein 3-Tupel an Oktal-Zahlen definiert.

[16] Beliebige Kombinationen der Permission-Bits (**rwx**) sind natürlich Möglich

Access Control
Anpassungen mit `chmod`

Typische Nutzung:

- `chmod 700 <file/directory>` (*Owner* darf alles, *Group* und *Other* nichts)
- `chmod +x <file/directory>` (*Jeder* hat **execute** Rechte)
- `chmod -x <file/directory>` (*Niemand* hat **execute** Rechte)

Access Control
Anpassungen mit `chmod`

Typische Nutzung:

- `chmod 700 <file/directory>` (*Owner* darf alles, *Group* und *Other* nichts)
- `chmod +x <file/directory>` (*Jeder* hat **execute** Rechte)
- `chmod -x <file/directory>` (*Niemand* hat **execute** Rechte)

Der Owner kann sich alle Rechte wegnehmen dies aber auch *immer* Rückgängig machen.

Access Control
Anpassungen mit `chmod`

Typische Nutzung:

- `chmod 700 <file/directory>` (*Owner* darf alles, *Group* und *Other* nichts)
- `chmod +x <file/directory>` (*Jeder* hat **execute** Rechte)
- `chmod -x <file/directory>` (*Niemand* hat **execute** Rechte)

Der Owner kann sich alle Rechte wegnehmen dies aber auch *immer* Rückgängig machen.

Überprüfen Sie auf der *Rechnerhalle* die Rechte ihres Home-Verzeichnis (`~`[a]) mit z. B. `stat ~`. Idealeweiße sollten nur Sie alleine Rechte auf dieses Verzeichnis haben!

[a] Auf der Rechnerhalle ist das `/u/halle/<username>/home_at/`

Text/Code Editing

Editors



xkcd 1823

Text Editors und IDE's

Text Editors sind leichtgewichtige Programme zum editieren von Text und Code:

- nano
- vi, vim, neovim
- Sublime Text
- Visual Studio Code, VSCodium
- Zed

Text Editors und IDE's

Text Editors sind leichtgewichtige Programme zum editieren von Text und Code:

- nano
- vi, vim, neovim
- Sublime Text
- Visual Studio Code, VSCodium
- Zed

IDE's (*Integrated Development Environment*) bündeln zusätzliche Tools:

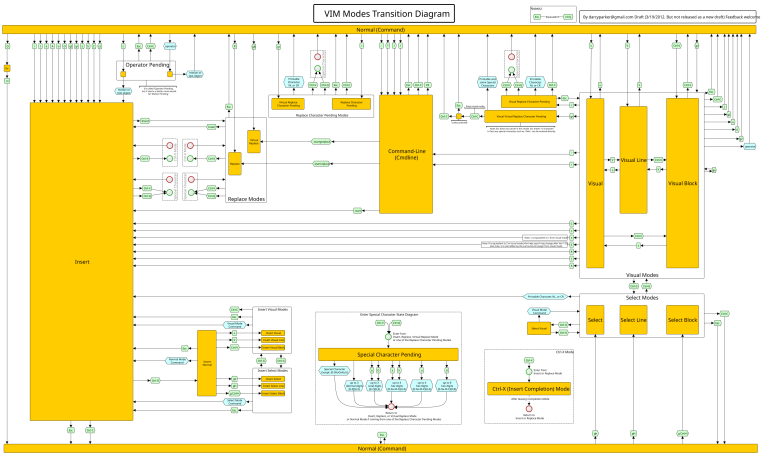
- IntelliJ
- Eclipse
- Visual Studio

Terminal Editoren
nano

- Einfach in der Bedienung
- In der Kurzanleitung von *nano* ist mit `^` die Taste **Strg** gemeint
- Mit `^X` (**Strg+X**) wird *nano* beendet
- Mit `^O` (**Strg+O**) wird der modifizierte Inhalt gespeichert

Terminal Editoren

vim



vim als State-Machine

Terminal Editoren

`vim`

In `vim` wird mit „Modes“ gearbeitet (z.B. **INSERT-MODE**, **NORMAL-MODE** und **CMD-LINE**):

- **NORMAL-MODE** (Startzustand):
  - Navigation im Text (z.B. `h`, `j`, `k`, `l`, `gg`, `G`)
  - `i` ➔ **INSERT-MODE**
  - `:` ➔ **CMD-LINE**
- **INSERT-MODE**:
  - Normale Texteingabe am cursor
  - *ESC* ➔ **NORMAL-MODE**
- **CMD-LINE**:
  - `:q`—„Quit“ (beendet vim)
  - `:w`—„Write“ (speichert den editierten text)
  - `:wq`—„Write and Quit“
  - `<command>!` versucht das Command zu erzwingen
  - *ESC* ➔ **NORMAL-MODE**

Basic Scripting

Basic Scripting

Durch *Scripting* kann die Ausführung mehrer Commands in einer Datei festgelegt werden.

Komponenten eines Scripts:

1. **shebang**: `#!` + Vollständiger Pfad zum Interpreter (z. B. sh[17], bash, zsh, fish, …)
2. Die auszuführenden Commands

[17] Die Scripte die wir besprechen sind POSIX compliant mit sh als Interpreter

Basic Scripting

Durch *Scripting* kann die Ausführung mehrer Commands in einer Datei festgelegt werden.

Komponenten eines Scripts:

1. **shebang**: `#!` + Vollständiger Pfad zum Interpreter (z. B. sh[17], bash, zsh, fish, …)
2. Die auszuführenden Commands

```
#!/bin/sh

echo "My first Script"
```

[17] Die Scripte die wir besprechen sind POSIX compliant mit sh als Interpreter

Basic Scripting

Durch *Scripting* kann die Ausführung mehrer Commands in einer Datei festgelegt werden.

Komponenten eines Scripts:

1. **shebang**: `#!` + Vollständiger Pfad zum Interpreter (z. B. sh[17], bash, zsh, fish, …)
2. Die auszuführenden Commands

```
#!/bin/sh

echo "My first Script"
```

Die Dateiendung eines Scripts (für Shell-Scripte typischerweiße `.sh`) dient nur zur Unterscheidung! Die **shebang** alleine ist ausschlaggebend für den ausgewählten Interpreter.

[17] Die Scripte die wir besprechen sind POSIX compliant mit sh als Interpreter

Basic Scripting

Scripte sind nicht limitiert auf Shells. Python ist z. B. auch eine Script Sprache!

Python Script

```python
#!/bin/python3

name = input("Name?\n")
print(f"Hi {name}")
```

Shell Script

```sh
#!/bin/sh

read -p "Name?\n" name
echo "Hi $name"
```

Basic Scripting

Scripte sind nicht limitiert auf Shells. Python ist z. B. auch eine Script Sprache!

Python Script

```python
#!/bin/python3

name = input("Name?\n")
print(f"Hi {name}")
```

Shell Script

```sh
#!/bin/sh

read -p "Name?\n" name
echo "Hi $name"
```

Damit ein Script ausgeführt werden kann, muss das **x** Bit gesetzt sein (`chmod`)! Danach kann das Script wie jedes andere Programm gestartet werden.

Basic Scripting
Variablen

Zuordnung von Text bzw. Zahlen auf eine Variable (*Identifier*) mit `var=value` (*ohne spaces*):

```
#!/bin/sh
name="Yuuko"
age=16
```

[18] Der Variablen-Zugriff (inklusive $) kann gequoted werden um Probleme mit enthaltenen Leerzeichen zu vermeiden

Basic Scripting
Variablen

Zuordnung von Text bzw. Zahlen auf eine Variable (*Identifier*) mit `var=value` (*ohne spaces*):

```
#!/bin/sh
name="Yuuko"
age=16
```

Auf Variablen wird mit `$` + *Identifier* (oder `$` + `{` + *Identifier* + `}`) zugegriffen[18]:

```
echo "Wie alt ist $name; $name ist ${age} Jahre; Mogami River"
```

[18] Der Variablen-Zugriff (inklusive $) kann gequoted werden um Probleme mit enthaltenen Leerzeichen zu vermeiden

Basic Scripting
Variablen

Zuordnung von Text bzw. Zahlen auf eine Variable (*Identifier*) mit `var=value` (*ohne spaces*):

```
#!/bin/sh
name="Yuuko"
age=16
```

Auf Variablen wird mit `$` + *Identifier* (oder `$` + `{` + *Identifier* + `}`) zugegriffen[18]:

```
echo "Wie alt ist $name; $name ist ${age} Jahre; Mogami River"
```

Manche commands definieren und belegen eine Variable gleichzeitig (z. B. `read`)

[18] Der Variablen-Zugriff (inklusive $) kann gequoted werden um Probleme mit enthaltenen Leerzeichen zu vermeiden

Basic Scripting

if then else

- `if [ condition ]; then ... fi`
- `if [ condition ]; then ... else ... fi` (mehrere Cond-Branches mit `elif`)

Basic Scripting

if then else

- `if [ condition ]; then ... fi`
- `if [ condition ]; then ... else ... fi` (mehrere Cond-Branches mit `elif`)

Für `condition` entweder:

- Belegung einer Variable überprüfen (`=` bzw. `!=`).
- *POSIX File Tests* benutzen:

Shell Test	Beschreibung
`-e file`	Existiert `file`?
`-f file`	Ist `file` eine reguläre Datei?
`-d file`	Ist `file` ein Verzeichnis?
`-r file`	Ist `file` readable?
`-w file`	Ist `file` schreibbar?
`-x file`	Ist `file` ausführbar?

Basic Scripting
if then else

```
#!/bin/sh

read -p "Enter Filename" file

# Note: spaces around = and quotes around $file are important!
if [ "$file" = "secret" ] && [ -f "$file" ]; then
    echo "This file looks confidential!"
elif [ -f "$file" ]; then
    echo "File exists!"
else
    echo "File not found!"
fi
```

Basic Scripting
for loops

- `for variable in list; do ... done`

Für `var` wird ein beliebiger Identifier eingesetzt der die Elemente von `list` bekommt.

[19] Auf ein bestimmtes positional Argument kann mit `${<index>}` zugegriffen werden

Basic Scripting
for loops

- `for variable in list; do ... done`

Für `var` wird ein beliebiger Identifier eingesetzt der die Elemente von `list` bekommt.

`list` kann u. A.:

- hardcoded sein: z. B. `1 2 3`

[19] Auf ein bestimmtes positional Argument kann mit `${<index>}` zugegriffen werden

Basic Scripting
for loops

- `for variable in list; do ... done`

Für `var` wird ein beliebiger Identifier eingesetzt der die Elemente von `list` bekommt.

`list` kann u. A.:

- hardcoded sein: z. B. `1 2 3`
- das Resultat von *Globbing* sein: z. B. `*.txt`
  ➔ Alle Dateien im *CWD* die in `.txt` enden

[19] Auf ein bestimmtes positional Argument kann mit `${<index>}` zugegriffen werden

Basic Scripting
for loops

- `for variable in list; do ... done`

Für `var` wird ein beliebiger Identifier eingesetzt der die Elemente von `list` bekommt.

`list` kann u. A.:

- hardcoded sein: z. B. `1 2 3`
- das Resultat von *Globbing* sein: z. B. `*.txt`
  ➔ Alle Dateien im *CWD* die in `.txt` enden
- das Resultat von *Command Substitution* sein: `$(command)` (z. B. `$(cat file)`)
  ➔ Alle Zeilen der Ausgabe von `command`

[19] Auf ein bestimmtes positional Argument kann mit `${<index>}` zugegriffen werden

Basic Scripting
for loops

- `for variable in list; do ... done`

Für `var` wird ein beliebiger Identifier eingesetzt der die Elemente von `list` bekommt.

`list` kann u. A.:

- hardcoded sein: z. B. `1 2 3`
- das Resultat von *Globbing* sein: z. B. `*.txt`
  ➔ Alle Dateien im *CWD* die in `.txt` enden
- das Resultat von *Command Substitution* sein: `$(command)` (z. B. `$(cat file)`)
  ➔ Alle Zeilen der Ausgabe von `command`
- die CLI args sein: default oder explizit mit `$@`[19]

[19] Auf ein bestimmtes positional Argument kann mit `${<index>}` zugegriffen werden

Basic Scripting
for loops

```
#!/bin/sh

echo "supercat!"
read -p "enter file globbing pattern: " glob

for file in *"$glob"; do
    if [ -f "$file" ]; then
        echo "> Content of ${file}:"
        cat "$file"
    fi
done
```

Basic Scripting



xkcd 1654

Nützliche Commands

Nützliche Commands | `grep`

Steht für „**g**lobal **r**egular **e**xpression search and **p**rint“.

`grep` sucht Muster in Dateien mit einem *RegEx* (Regular Expression)[20].

Typische Nutzung:

- Generell: `grep "RegEx" <File/File-Globbing>`
- `grep "test" test.txt`—Ausgabe aller Zeilen von `test.txt` die „test“ enthalten
- `cat *.txt | grep "test"`—Häufige Nutzung auch mit `|`
- Nützliche Optionen sind `-c` (für „count“) und `-i` für „ignore-case“

`ripgrep` (Command: `rg`) ist eine alternative Implementation in Rust ähnlich zu `grep`.

[20] Mehr dazu in der Vorlesung „Theoretische Informatik“

Nützliche Commands | wc

Steht für „word count“.

Gibt die Anzahl der in einer Datei enthaltenen Zeilen, Wörter und Bytes aus.

Typische Nutzung:

- `wc test.txt`

Nützliche Commands | `ping`

Sendet eine *ICMP ECHO_REQUEST*[21] an die Angegebene *IP-Adresse* bzw. *URL*.

Nützlich um die Verfügbarkeit eines Services über das Netzwerk zu überprüfen.

Typische Nutzung:

- `ping zulip.in.tum.de`

[21] Mehr dazu in der Vorlesung „Grundlagen Rechnernetze“

Nützliche Commands | scp

Steht für „*OpenSSH* secure file copy“

Kopieren von Dateien zwischen zwei Rechnern über ein Netzwerk.

Typische Nutzung:

- `scp path/to/local/file <username>@<remote-URL>:path/to/remote/copy`
- z.B. `scp test.txt <ito-username>@lxhalle.in.tum.de:~/`

Nützliche Commands | `htop` und `btop`

„Task Manager“ von Linux.

Typische Nutzung:

- `htop`

Moderne Alternative: `btop`

Nützliche Commands | `more`

Der Inhalt langer Dateien oder Output von einem Command wird „Seitenweise“ angezeigt.

Typische Nutzung:

- `cat huge-file | more`
- `more huge-file`

Nützliche Commands | `less`

`less` ist ähnlich zu `more`, kann aber wesentlich mehr!

- `less` muss nicht die gesamte Datei eingelesen haben ➔ startet schneller
- Durch die Datei kann Zeilenweise gescrollt werden
- …

Typische Nutzung:

- `cat huge-file | less`
- `less huge-file`

Nützliche Commands | `alias`

Erstellt ein Alias für ein Nutzerdefiniertes command.

Beispiel:

- `alias wisdom="fortune | cowsay"`—Nun kann `wisdom` als command benutzt werden

Ein Nutzerdefiniertes Alias bleibt nur für eine Session bestehen![22]

[22] Ein persistentes Alias wird in Konfigurationsdateien (z.B. `~/.bashrc` für *bash* oder `~/.zshrc` für *zsh*) definiert

Nützliche Commands | `history`

Zeigt die vorhande history and eingegebenen commands an.

Typische Nutzung:

- `history`

Nützliche Commands | `tree`

Zeigt den Filesystembaum ausgehend vom *CWD* an.

Typische Nutzung:

- `tree`

Nützliche Commands | `pushd` und `popd`

Steht für „push directory“ und „pop directory“.

Erleichtern die Navigation durch die Möglichkeit Dateipfade zwischenzuspeichern.

Typische Nutzung:

1. `pushd /bin`—Speichert das *CWD* und wechselt zu dem Ordner `/bin`
2. `popd`—Ändert das aktuelle *CWD* zu dem vorher gespeichertem